St. Francis Institute of Technology, Mumbai-400 103
Department Of Information Technology

A.Y. 2025-2026
Class: BE-ITA/B, Semester: VII
Subject: Data Science Lab

Experiment – 5

1. **Aim:** To design a simple LSTM Network.

2. **Objectives:** Students should be familiarize with Learning Architectures and Frameworks using LSTM and CNN.
3. **Prerequisite:** Python basics

4. **Pre-Experiment Exercise:**
Theory:
Long Short Term Memory Network(LSTM). In concept, an LSTM recurrent unit tries to "remember" all the past knowledge that the network is seen so far and to "forget" irrelevant data. This is done by introducing different activation function layers called "gates" for different purposes. Each LSTM recurrent unit also maintains a vector called the Internal Cell State which conceptually describes the information that was chosen to be retained by the previous LSTM recurrent unit. A Long Short Term Memory Network consists of four different gates for different purposes as described below:-

Forget Gate(f): It determines to what extent to forget the previous data.
Input Gate(i): It determines the extent of information be written onto the Internal Cell State.
Input Modulation Gate(g): It is often considered as a sub-part of the input gate and much literature on LSTM's does not even mention it and assume it is inside the Input gate. It is used to modulate the information that the Input gate will write onto the Internal State Cell by adding non-linearity to the information and making the information Zero-mean. This is done to reduce the learning time as Zero-mean input has faster convergence. Although this gate's actions are less important than the others and are often treated as a finesse-providing concept, it is good practice to include this gate in the structure of the LSTM unit.
Output Gate(o): It determines what output(next Hidden State) to generate from the current Internal Cell State.

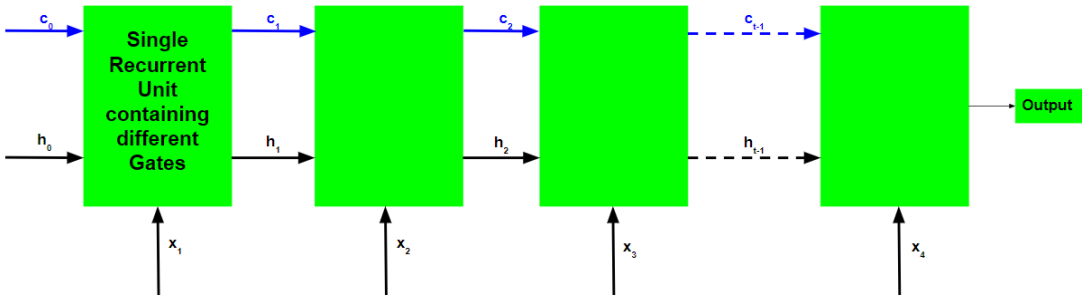


Fig:6.1 LSTM Network

CNN:
A convolutional neural network (CNN, or ConvNet) is a class of artificial neural network (ANN), most commonly applied to analyze visual imagery. CNNs are also known as Shift Invariant or Space Invariant Artificial Neural Networks (SIANN), based on the shared-weight architecture of the convolution kernels or filters that slide along input features and provide translation-equivariant responses known as feature maps. Counter-intuitively, most convolutional neural networks are not invariant to translation, due to the down sampling operation they apply to the input. They have applications in image and video recognition, recommender systems, image classification, image segmentation, medical image analysis,

natural language processing, brain–computer interfaces, and financial time series.

6. **Laboratory Exercise**

Procedure

- Take input the current input, the previous hidden state, and the previous internal cell state.
- Calculate the values of the four different gates by following the below steps:-
- For each gate, calculate the parameterized vectors for the current input and the previous hidden state by element-wise multiplication with the concerned vector with the respective weights for each gate.
- Apply the respective activation function for each gate element-wise on the parameterized vectors.
- Calculate the current internal cell state by first calculating the element-wise multiplication vector of the input gate and the input modulation gate, then calculate the element-wise multiplication vector of the forget gate and the previous internal cell state and then adding the two vectors.
- c_{t} = i\odot g + f\odot c_{t-1}

```
import numpy as np
def sigmoid(x):
    return 1 / (1 + np.exp(-x))
def tanh(x):
    return np.tanh(x)
# Initialize input, previous hidden and cell states (example dimensions)
input_dim = 3
hidden_dim = 3
# Random example inputs
x_t = np.array([0.5, 0.1, -0.3])        # current input
h_prev = np.array([0.2, -0.1, 0.4])        # previous hidden state
c_prev = np.array([0.1, 0.3, -0.2])      # previous cell state
# Weight vectors for gates (element-wise weights for input and hidden)
W_x_i = np.array([0.7, -0.4, 0.3])     # weights for input gate, input
W_h_i = np.array([0.5, 0.1, -0.6])     # weights for input gate, hidden
b_i = np.array([0.0, 0.0, 0.0])      # bias for input gate
W_x_f = np.array([0.6, 0.2, -0.3])     # weights for forget gate, input
W_h_f = np.array([-0.1, 0.4, 0.7])     # weights for forget gate, hidden
b_f = np.array([0.0, 0.0, 0.0])        # bias for forget gate
W_x_o = np.array([-0.5, 0.3, 0.8])     # weights for output gate, input
W_h_o = np.array([0.2, -0.3, 0.5])     # weights for output gate, hidden
b_o = np.array([0.0, 0.0, 0.0])        # bias for output gate
W_x_g = np.array([0.4, -0.6, 0.1])     # weights for input modulation gate, input
W_h_g = np.array([-0.2, 0.7, 0.3])     # weights for input modulation gate, hidden
b_g = np.array([0.0, 0.0, 0.0])        # bias for input modulation gate
# 1. Calculate gate activations (element-wise multiplication and addition)
i = sigmoid(W_x_i * x_t + W_h_i * h_prev + b_i)  # input gate
f = sigmoid(W_x_f * x_t + W_h_f * h_prev + b_f)  # forget gate
o = sigmoid(W_x_o * x_t + W_h_o * h_prev + b_o)  # output gate
g = tanh(W_x_g * x_t + W_h_g * h_prev + b_g)     # input modulation gate
# 2. Update cell state
c_t = f * c_prev + i * g
# 3. Compute current hidden state
h_t = o * tanh(c_t)
print("Input gate i:", i)
print("Forget gate f:", f)
print("Output gate o:", o)
print("Input modulation gate g:", g)
print("Updated cell state c_t:", c_t)
print("Updated hidden state h_t:", h_t)
```

Output:

```
Input gate i: [0.61063923 0.4875026  0.41824062]
Forget gate f: [0.56954622 0.49500017 0.59145898]
Output gate o: [0.44769209 0.5149955  0.49000133]
Input modulation gate g: [ 0.1586485  -0.12927258  0.08975778]
Updated cell state c_t: [ 0.15383162  0.08547933 -0.08075144]
Updated hidden state h_t: [ 0.06833105  0.04391456 -0.03948253]
```

Post-Experiments Exercise:

A. Extended Theory:

a. Explain Architecture of CNN.

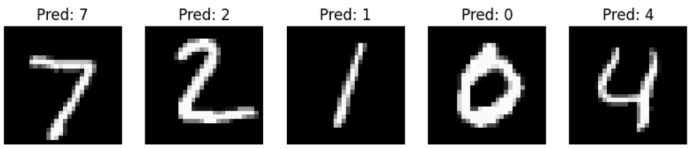
B. Post Lab Program:

a. Write a Python program to demonstrate any real life application using CNN

```
import tensorflow as tf
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt

# Load MNIST dataset (handwritten digits)
(X_train, y_train), (X_test, y_test) =
tf.keras.datasets.mnist.load_data()

# Normalize pixel values to [0,1]
X_train = X_train / 255.0
X_test = X_test / 255.0

# Reshape data to add channel dimension (needed for
CNN)
X_train = X_train.reshape(-1, 28, 28, 1)
X_test = X_test.reshape(-1, 28, 28, 1)

# Build the CNN model
model = models.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu',
input_shape=(28, 28, 1)),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')  # 10 classes for digits
0-9
])
# Compile the model
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
# Train the model
model.fit(X_train, y_train, epochs=5, validation_split=0.1)
# Evaluate the model on test data
test_loss, test_acc = model.evaluate(X_test, y_test)
print(f"Test accuracy: {test_acc:.4f}")
# Predict on some test images and display results
import numpy as np
predictions = model.predict(X_test[:5])
predicted_labels = np.argmax(predictions, axis=1)
plt.figure(figsize=(10, 2))
for i in range(5):
    plt.subplot(1, 5, i + 1)
    plt.imshow(X_test[i].reshape(28, 28), cmap='gray')
    plt.title(f"Pred: {predicted_labels[i]}")
    plt.axis('off')
plt.show()
```

Output:

```
Epoch 1/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 61s 35ms/step - accuracy: 0.8803 - loss: 0.3677 - val_accuracy: 0.9833 - val_loss: 0.0553
Epoch 2/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 81s 34ms/step - accuracy: 0.9839 - loss: 0.0511 - val_accuracy: 0.9872 - val_loss: 0.0437
Epoch 3/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 50s 29ms/step - accuracy: 0.9890 - loss: 0.0354 - val_accuracy: 0.9890 - val_loss: 0.0392
Epoch 4/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 79s 27ms/step - accuracy: 0.9919 - loss: 0.0246 - val_accuracy: 0.9910 - val_loss: 0.0343
Epoch 5/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 47s 28ms/step - accuracy: 0.9943 - loss: 0.0171 - val_accuracy: 0.9875 - val_loss: 0.0489
313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - accuracy: 0.9850 - loss: 0.0477
Test accuracy: 0.9892
WARNING:tensorflow:5 out of the last 5 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 105ms/step
```



C. Conclusion:

1. Write what was performed in the program (s) .
2. What is the significance of program and what Objective is achieved?

D. References:

[1] https://stackabuse.com/image-recognition-in-python-with-tensorflow-and-keras/
[2] . https://www.simplilearn.com/tutorials/deep-learning-tutorial/rnn